



Demo Scenarios Study Guide

The nine scenarios OptiDBX can demonstrate — what each one proves, the measured before/after, and exactly what to say when you present it.

Project: OptiDBX — Safe PostgreSQL Auto-Tuning

Document: 1 of the OptiDBX Study Series

Source of numbers: `dashboard/src/scenarios.mjs` → `docs/optidbx_technical_report.md`

Prepared: 22 September 2026

THE OPTIDBX STUDY SERIES — READ THESE TOGETHER

- | | | |
|----------|---|---------------------|
| 1 | Demo Scenarios Study Guide
Understand every scenario the dashboard can demonstrate. | YOU ARE HERE |
| 2 | Viva Question Bank
Questions a panel may ask, with grounded answers. | |
| 3 | Technical Terms Glossary
Every term in the project, defined plainly. | |
| 5 | Page-by-Page Presentation Scripts
What to say on each page of the dashboard. | |

The shared foundation

Every scenario below is the same machine seen under different conditions. Read this once; then each scenario just varies the inputs.

One knob. OptiDBX tunes a single PostgreSQL parameter, `max_parallel_workers_per_gather`, and only ever moves it between the approved values **1, 2, 4, 6, 8** — one step at a time. A narrow, reversible action is what makes acting on a live database safe.

The six-stage decision pipeline

STAGE	PURPOSE
1. Baseline	Collect a warm, steady-state reading of the owned workload. No change yet.
2. Detection	Require three consecutive problematic readings before acting — a single spike is ignored.
3. Apply	Journal the action to disk, then reduce parallelism one approved step and verify the new value.
4. Observation	Measure the effect on the owned workload itself — client p95 latency and throughput.
5. Decision	Apply the net-benefit rule: keep only a measured improvement; otherwise revert.
6. Report	Produce a before/after report with provenance and honest caveats.

The net-benefit rule. A change is kept only if the **owned workload** — the queries OptiDBX itself is running — shows a real improvement in client-observed **p95 latency** and **throughput**, with no latency regression. It is judged on what the user feels, not on noisy database-wide counters.

The five possible verdicts

VERDICT	MEANING
KEEP	The change measurably helped and cleared the net-benefit threshold, so it was kept.
ROLLBACK	The change measurably hurt, so the original value was restored with a verified rollback.
NO_ACTION	No sustained contention was confirmed, so nothing was changed — correct restraint.
RECOMMENDATION	A condition outside the safe automatic scope was surfaced for an operator, with evidence.
RECOVERY	A rollback could not be verified, so the tuner fails closed and blocks further actions.

How to read each scenario card

- **Tags** show the category (CPU / DB / OS), whether it is **Simulated** or **Live**, and the expected verdict.
- **Pipeline walkthrough** traces the six stages with the telemetry at each step.
- **Before/after bars** — grey is *before*, orange is *after*; the chip shows the percentage change (orange = improvement).
- **Presenter script** is a line you can say out loud; **Connects to** points to the companion documents.

PART B

The nine scenarios

Grouped by what they prove: that the tuner acts when it should, reverts when it must, and shows restraint when nothing is wrong.

Success — a change is kept (4)

A genuine bottleneck is found and one safe step delivers a measured improvement.

1 CPU / Parallelism Contention

CPU

SIMULATED

Change kept

What it demonstrates. The core success case — a real bottleneck is found, one safe step is applied, and the measured improvement earns a KEEP.

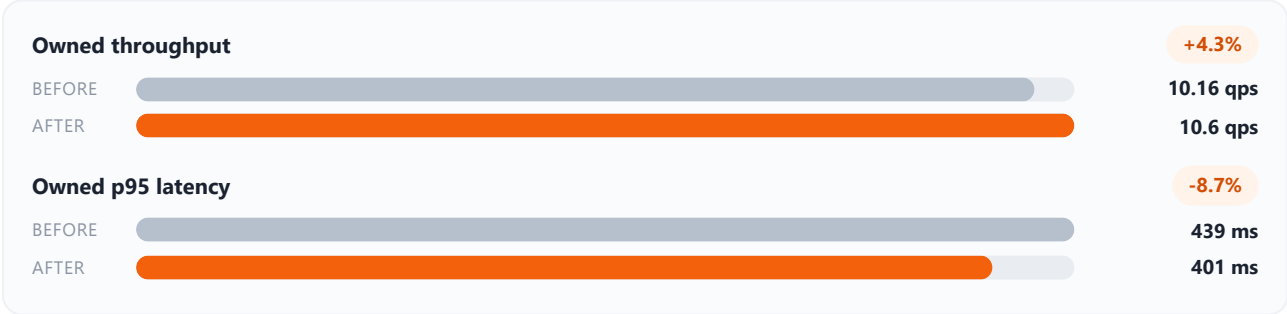
THE SITUATION

An over-parallelized server is under heavy analytical load. CPU sits at ~94%, ~30 parallel workers are active, context switches are high and query latency is climbing.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Baseline	Warm baseline collected in recommendation mode.	CPU 94% · 30 workers · 280 ms
Detection	Three consecutive readings confirm sustained contention.	CPU 95% · 31 workers · 293 ms
Apply	Journalled, then max_parallel_workers_per_gather 8 → 6, verified.	CPU 93% · 24 workers · 270 ms
Observation	Owned workload measured for ~30 s (client p95 & QPS).	CPU 92% · 22 workers · 250 ms
Decision	Throughput +4.3%, tail latency -8.8%: net benefit clears threshold.	CPU 92% · 22 workers · 245 ms

OUTCOME & EVIDENCE — MAX_PARALLEL_WORKERS_PER_GATHER 8 → 6 (KEPT)



WHY IT MATTERS

Shows the whole point of the tool: it does not guess — it applies one reversible step and keeps it only because the owned-workload tail latency genuinely fell.

PRESENTER SCRIPT

"Here the server is drowning in parallel workers. OptiDBX steps parallelism down once, measures a real 8.8% drop in tail latency, and keeps the change."

CONNECTS TO

Glossary (Doc 3): max_parallel_workers_per_gather, p95 latency, net-benefit rule, context switches.
Likely viva (Doc 2): "Why reduce parallelism to make a database faster?"

2 Analytical Reporting Peak

DB

SIMULATED

Change kept

What it demonstrates. The same success path under a different trigger — a burst of dashboard/report queries that saturates the CPU.

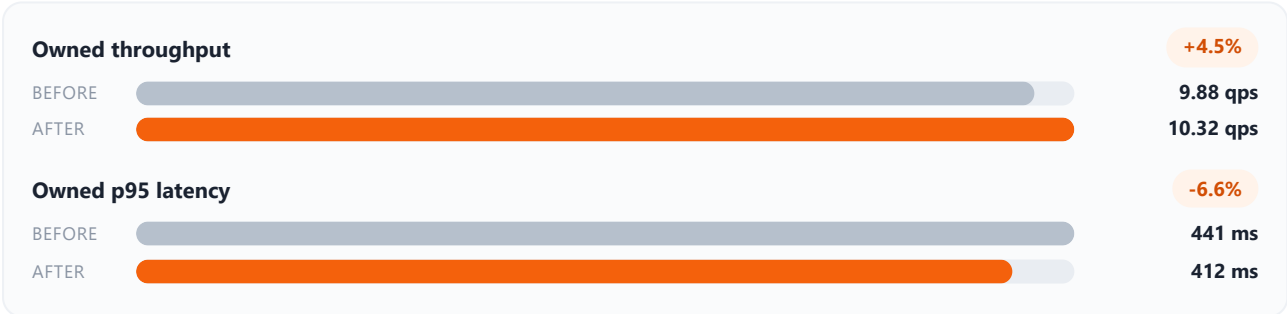
THE SITUATION

Many reporting queries arrive together during a refresh storm. Every worker slot is busy and latency keeps rising.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Baseline	Warm baseline captured during the peak.	CPU 92% · 28 workers · 300 ms
Detection	Latency climbs across three consecutive readings.	CPU 96% · 31 workers · 318 ms
Apply	8 → 6 on owned sessions only, journalled and verified.	CPU 94% · 23 workers · 300 ms
Observation	Queueing falls; queries complete more predictably.	CPU 91% · 21 workers · 268 ms
Decision	Throughput +4.5%, tail latency -6.6%: KEPT.	CPU 91% · 21 workers · 262 ms

OUTCOME & EVIDENCE — MAX_PARALLEL_WORKERS_PER_GATHER 8 → 6 (KEPT)



WHY IT MATTERS

Proves the behaviour generalises beyond one workload shape: a reporting peak is absorbed with lower tail latency at slightly higher throughput.

PRESENTER SCRIPT

"A reporting storm hits. Same safe step, same honest measurement — 6.6% better tail latency, so we keep it."

CONNECTS TO

Glossary (Doc 3): throughput (QPS), p95 latency, owned workload.

Likely viva (Doc 2): "How does the tuner avoid acting only on the very peak second?"

3 Nightly Batch Window

DB

SIMULATED

Change kept

What it demonstrates. A sustained (not bursty) load — long overnight aggregations competing for cores.

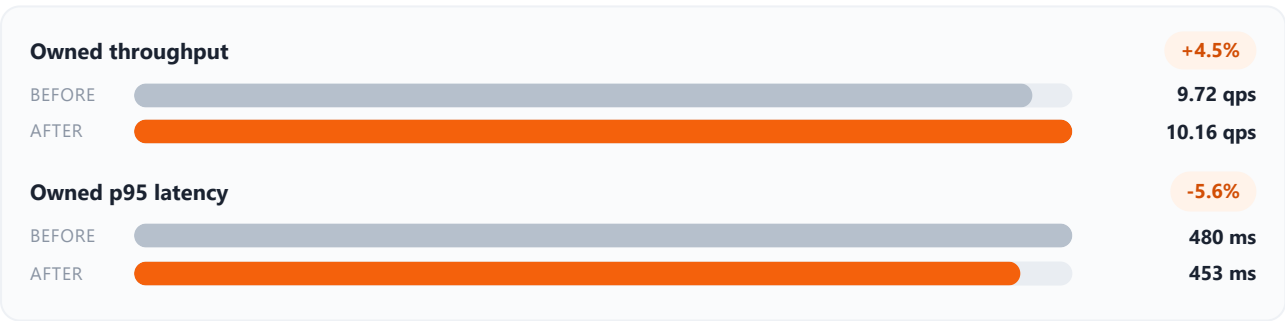
THE SITUATION

Heavy overnight aggregations pin the CPU for an extended period; individual aggregations fight each other for cores.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Baseline	Baseline taken after the batch warms up.	CPU 93% · 29 workers · 330 ms
Detection	CPU pinned and workers saturated for three intervals.	CPU 97% · 31 workers · 352 ms
Apply	One approved step down, applied and verified.	CPU 95% · 24 workers · 335 ms
Observation	Same aggregate pace; less core contention per query.	CPU 92% · 22 workers · 300 ms
Decision	Throughput +4.5%, tail latency -5.5%: KEPT.	CPU 92% · 22 workers · 296 ms

OUTCOME & EVIDENCE — MAX_PARALLEL_WORKERS_PER_GATHER 8 → 6 (KEPT)



WHY IT MATTERS

Distinguishes a sustained bottleneck from a transient one (contrast Scenario 7): sustained contention is exactly what the action targets.

PRESENTER SCRIPT

"The nightly batch keeps its pace, but the slowest aggregations get measurably faster. That earns a keep."

CONNECTS TO

Glossary (Doc 3): parallel workers, tail latency, warm-up exclusion.
Likely viva (Doc 2): "What is the difference between throughput and latency here?"

4 Second Step-Down

What it demonstrates. Cascading tuning done safely — after a first kept change, a second independent step from 6 to 4.

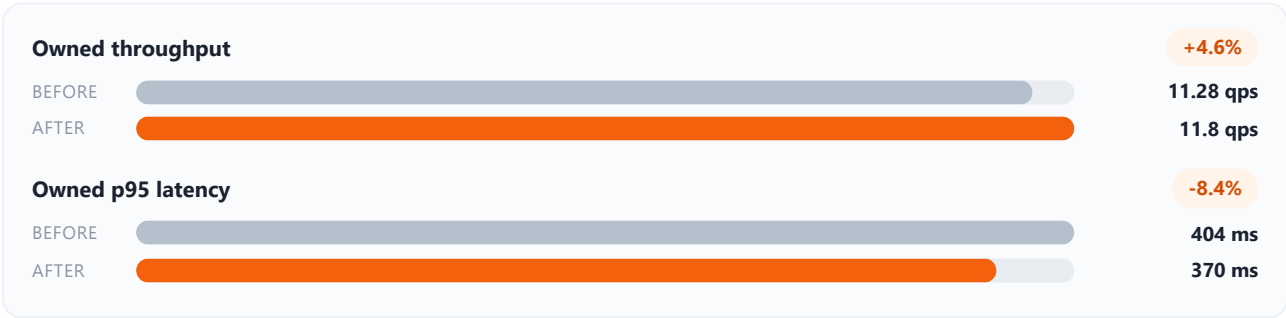
THE SITUATION

A previous change (to 6) is already in effect and cooldown has finished, but contention is still confirmed on a fresh baseline.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Baseline	Fresh baseline — evidence never carries over between actions.	CPU 93% · 24 workers · 295 ms
Detection	Contention re-confirmed over three new readings.	CPU 95% · 25 workers · 308 ms
Apply	6 → 4, the next approved value (never arbitrary).	CPU 93% · 18 workers · 290 ms
Observation	Second reduction measured on its own merits.	CPU 90% · 16 workers · 262 ms
Decision	Throughput +4.6%, tail latency -8.5%: KEPT.	CPU 90% · 16 workers · 256 ms

OUTCOME & EVIDENCE — MAX_PARALLEL_WORKERS_PER_GATHER 6 → 4 (KEPT)



WHY IT MATTERS

Shows the tuner can improve step by step without stacking evidence — each action gets its own fresh baseline, one change at a time.

PRESENTER SCRIPT

“Contention persists, so we take one more approved step to 4 — judged against its own fresh baseline, not the first change's.”

CONNECTS TO

Glossary (Doc 3): cooldown, approved values, fresh baseline.
Likely viva (Doc 2): “Why not jump straight from 8 to 4 in one move?”

Safety — a change is reverted (1)

The change hurt, so it is undone with a verified rollback.

5 Change Made It Worse

What it demonstrates. The safety net — a change is applied, the observation shows a regression, and it is reverted.

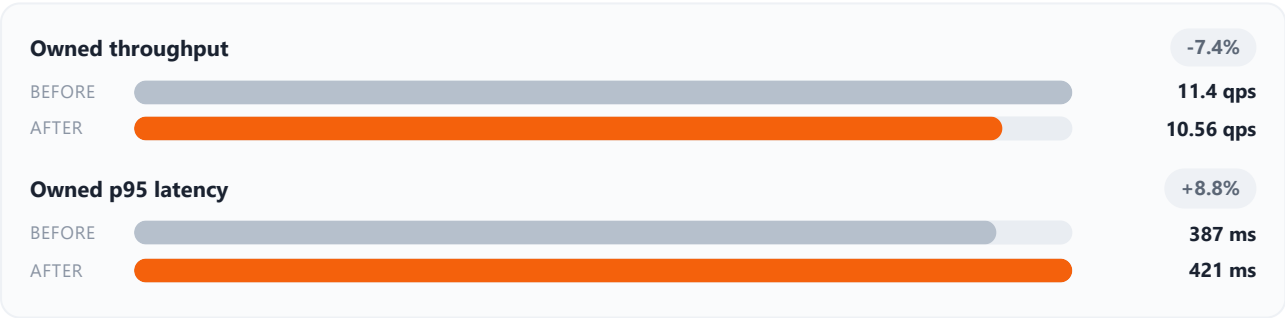
THE SITUATION

Contention is confirmed and a reduction is applied, but under observation the owned workload degrades instead of improving.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Baseline	Warm baseline on the owned workload.	CPU 91% · 28 workers · 260 ms
Detection	Contention confirmed; a reduction is recommended.	CPU 93% · 30 workers · 275 ms
Apply	Journalled, applied 8 → 6, verified.	CPU 92% · 24 workers · 280 ms
Observation	Owned workload gets worse — throughput down, latency up.	CPU 93% · 24 workers · 300 ms
Decision	Throughput -7.4%, tail latency +8.7%: ROLLED BACK.	CPU 92% · 28 workers · 285 ms

OUTCOME & EVIDENCE — REVERTED TO MAX_PARALLEL_WORKERS_PER_GATHER 8



WHY IT MATTERS

This is the credibility scenario: the tuner keeps changes only when they help. When a change backfires, the verified rollback restores the original setting.

PRESENTER SCRIPT

"This time the change hurt — throughput fell 7.4%. The safety net catches it and reverts, verified. That's why you can trust the keeps."

CONNECTS TO

Glossary (Doc 3): verified rollback, net-benefit rule, regression guard.
Likely viva (Doc 2): "How do you know a rollback actually restored the old value?"

Restraint — no change is made (2)

Nothing warranted a change; doing nothing is the correct answer.

6 Well-Provisioned Server

CPU

LIVE

No change required

What it demonstrates. Restraint on a healthy database — and this one runs live against the real backend.

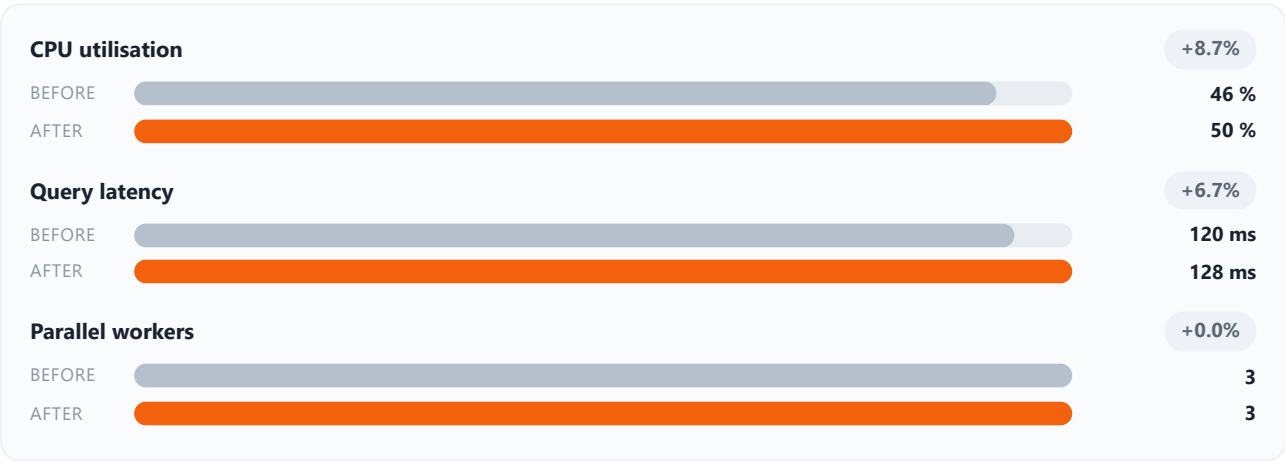
THE SITUATION

A healthy, non-oversubscribed database. CPU has headroom and only a few parallel workers are active.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Baseline	Real workload starts; telemetry streams in.	CPU 46% · 3 workers · 120 ms
Detection	No sustained contention; counter never reaches three.	CPU 52% · 3 workers · 130 ms
Decision	No bottleneck, no action — reported honestly.	CPU 50% · 3 workers · 128 ms

OUTCOME & EVIDENCE — NO CHANGE APPLIED



WHY IT MATTERS

A tuner that always changes something is dangerous. Doing nothing on a healthy system — and saying so — is the correct, honest outcome.

PRESENTER SCRIPT

“This one is live. The database is healthy, so OptiDBX does nothing — and tells you why. Restraint is a feature.”

CONNECTS TO

Glossary (Doc 3): NO_ACTION, oversubscription, recommendation mode.
Likely viva (Doc 2): “When is the right answer to change nothing?”

7 Transient Spike

OS

SIMULATED

No change required

What it demonstrates. Noise rejection — one bad reading is not a bottleneck.

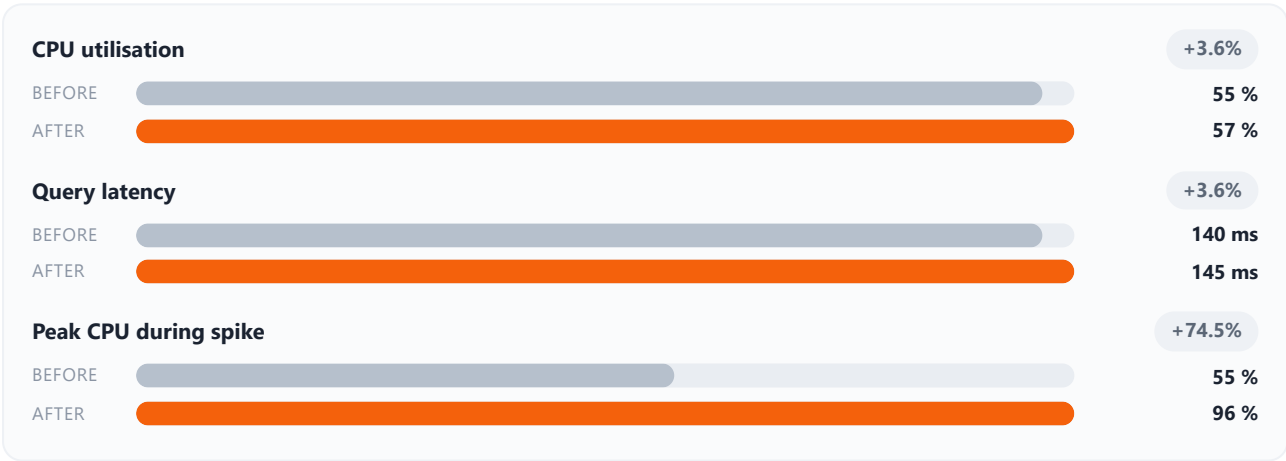
THE SITUATION

A momentary spike appears, then conditions return to normal within the next interval.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Baseline	A quiet baseline is collected.	CPU 55% · 4 workers · 140 ms
Detection	One interval spikes (1 of 3); next reading resets it to 0.	CPU 96% · 12 workers · 260 ms
Decision	Not sustained across three readings, so no action.	CPU 57% · 4 workers · 145 ms

OUTCOME & EVIDENCE — NO CHANGE APPLIED



WHY IT MATTERS

Demonstrates why the three-consecutive-reading rule exists: it prevents the tuner from overreacting to a single blip and thrashing the configuration.

PRESENTER SCRIPT

"CPU spikes for one second, then recovers. The three-reading rule refuses to act on noise — no change."

CONNECTS TO

Glossary (Doc 3): three-reading confirmation, debouncing, false positive.
Likely viva (Doc 2): "Why require three readings instead of acting immediately?"

Scope & recovery (2)

Problems outside the safe action set, and fail-closed behaviour when a rollback cannot be verified.

8 Memory Pressure

DB

SIMULATED

Recommendation only

What it demonstrates. Staying inside a safe scope — a real problem is detected but is not one the tuner is approved to fix automatically.

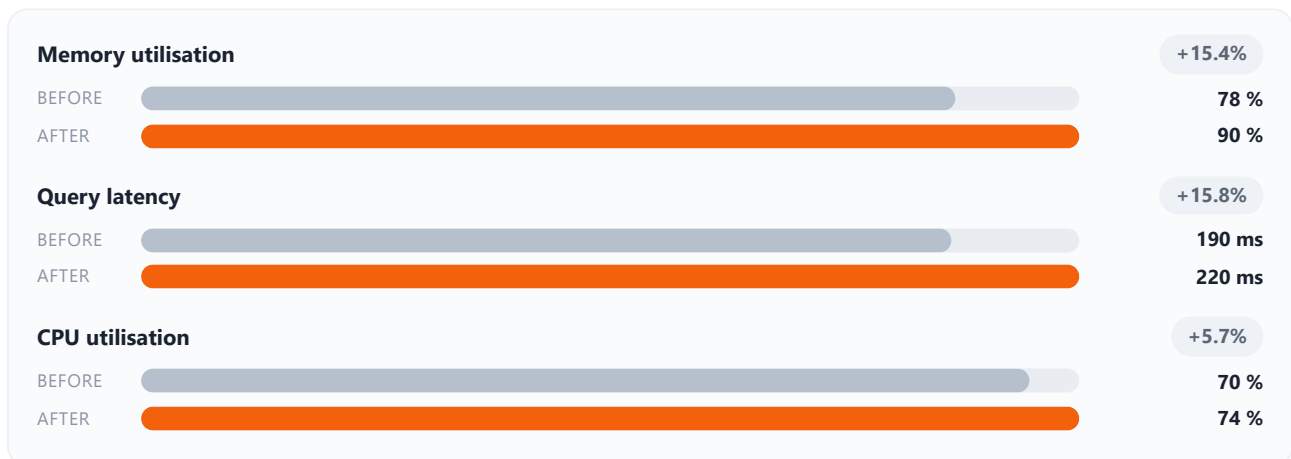
THE SITUATION

Memory utilisation climbs and temporary files spill to disk. This is a genuine issue, but memory/OS tuning is outside the approved automatic action set.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Baseline	Baseline collected as memory climbs.	CPU 70% · 6 workers · 190 ms · mem 78%
Detection	High memory and temp-file spill detected and explained.	CPU 74% · 6 workers · 220 ms · mem 90%
Decision	Out of safe scope — surfaced as an operator recommendation.	CPU 73% · 6 workers · 218 ms · mem 90%

OUTCOME & EVIDENCE — RECOMMENDATION ONLY — NO AUTOMATIC CHANGE



WHY IT MATTERS

Shows disciplined boundaries: the tuner explains a problem with evidence instead of reaching for a knob it was never approved to touch.

PRESENTER SCRIPT

"It clearly sees the memory pressure — but memory tuning isn't in its safe scope, so it hands the operator an explained recommendation instead of guessing."

CONNECTS TO

Glossary (Doc 3): RECOMMENDATION, safe action scope, temp-file spill.

Likely viva (Doc 2): "Why not let the tuner also fix memory settings?"

9 Rollback Failure → Recovery

DB

SIMULATED

Recovery engaged

What it demonstrates. The strongest safety guarantee — fail-closed behaviour when a rollback cannot be verified.

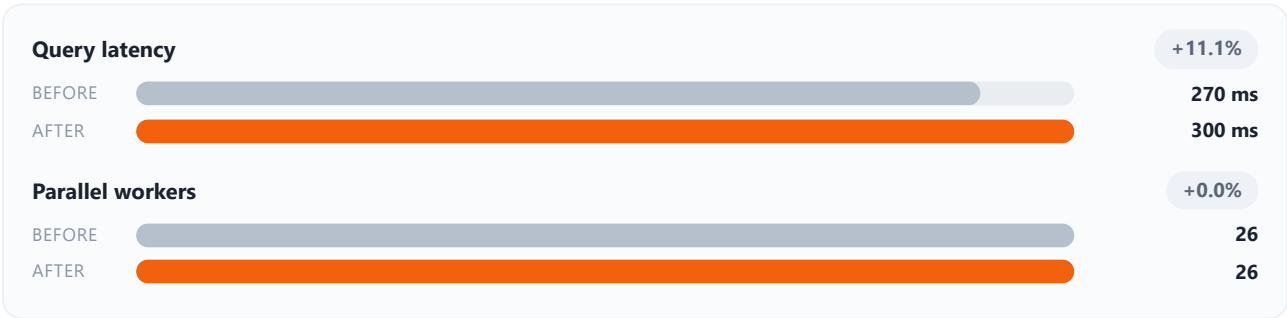
THE SITUATION

A change underperforms and a rollback is requested, but restoration cannot be verified.

PIPELINE WALKTHROUGH

STAGE	WHAT HAPPENS	KEY TELEMETRY
Apply	Change journalled to disk before any mutation.	CPU 92% · 26 workers · 270 ms
Observation	Change underperforms; rollback requested.	CPU 93% · 26 workers · 300 ms
Recovery	ROLLBACK_FAILED: keep original sessions, block all further actions.	CPU 93% · 26 workers · 300 ms

OUTCOME & EVIDENCE — FAIL-CLOSED — FURTHER ACTIONS BLOCKED PENDING AN OPERATOR



WHY IT MATTERS

The worst case is an ambiguous state. Here the durable journal stays authoritative, new connections cannot impersonate the recovery target, and the system refuses to do anything risky until a human resolves it.

PRESENTER SCRIPT

"If it can't even verify a rollback, it stops — keeps the original sessions, blocks new actions, and waits for a human. Fail-closed, never fail-dangerous."

CONNECTS TO

Glossary (Doc 3): fail-closed, recovery journal, ROLLBACK_FAILED.
Likely viva (Doc 2): "What happens if a rollback itself fails?"

All nine at a glance

A single table to revise from, and a suggested order for a live walk-through.

#	SCENARIO	CAT.	TYPE	VERDICT	ACTION
1	CPU / Parallelism Contention	CPU	Sim	Change kept	max_parallel_workers_per_gather 8 → 6 (kept)
2	Analytical Reporting Peak	DB	Sim	Change kept	max_parallel_workers_per_gather 8 → 6 (kept)
3	Nightly Batch Window	DB	Sim	Change kept	max_parallel_workers_per_gather 8 → 6 (kept)
4	Second Step-Down	CPU	Sim	Change kept	max_parallel_workers_per_gather 6 → 4 (kept)
5	Change Made It Worse	DB	Sim	Change reverted	Reverted to max_parallel_workers_per_gather 8
6	Well-Provisioned Server	CPU	Live	No change required	No change applied
7	Transient Spike	OS	Sim	No change required	No change applied
8	Memory Pressure	DB	Sim	Recommendation only	Recommendation only — no automatic change
9	Rollback Failure → Recovery	DB	Sim	Recovery engaged	Fail-closed — further actions blocked pending an operator

Suggested running order. Open with **1 (CPU / Parallelism Contention)** to show a clean keep, then **5 (Change Made It Worse)** so the audience sees the rollback that makes the keeps trustworthy, then **6 (Well-Provisioned, live)** to prove restraint on the real backend. Use 2–4 to show the keep generalises, and 7–9 if asked about noise, scope, or worst-case safety.

Honesty note. Scenarios marked **Simulated** are illustrative walk-throughs whose numbers are seeded from measured runs (see the Technical Report); the one marked **Live** measures the real PostgreSQL backend. A single before/after observation does not by itself prove causation or a general speedup.